

1. Introduction

The dataset used in this project focuses on analyzing NYC Yellow Taxi trip data for the year 2024. The dataset provides information about taxi trips, including trip distance, fare amount, passenger count, payment type, pickup and drop-off boroughs, and trip duration. This analysis helps identify patterns in taxi usage, payment methods, trip distances, fares, and travel behavior throughout the day.

2. About Dataset

The dataset used in this project focuses on NYC Yellow Taxi trip data for the year 2024. The dataset provides information about taxi trips, including trip distance, fare amount, passenger count, payment type, pickup and drop-off boroughs, and trip duration.

This dataset can be used to analyze taxi usage patterns, payment methods, trip distances, fares, passenger behavior, and travel patterns throughout the day.

3. Dataset Features

Date: Date on which the taxi trips occurred.

Hour: Hour of the day when the trips occurred.

Passenger Count: Number of passengers in the taxi.

Pickup Borough: Borough where the taxi trip started.

Drop-off Borough: Borough where the taxi trip ended.

Payment Type: Method of payment used for the trip.

Trip Count: Number of taxi trips.

Trip Distance: Total distance covered by the trips.

Duration: Total duration of the trips.

Fare Amount: Total fare amount generated from the trips.

Tip Amount: Total tip amount received.

Total Amount: Total amount collected from the trips.

4. Objectives

To understand the structure and characteristics of the NYC Yellow Taxi dataset.

To analyze taxi trip patterns based on different hours and locations.

To analyze the distribution of passengers and payment methods.

To study trip distance and fare amounts.

To identify patterns and relationships between different numerical variables.

To visualize important findings using simple data visualization techniques.

To derive useful insights from the taxi trip data.

5. Data Cleaning

Data cleaning was performed to ensure that the dataset was suitable for analysis. The dataset was checked for missing values and duplicate records. The data types of the columns were also examined to ensure that the values were stored in an appropriate format.

6. Feature Extraction

Feature extraction was performed to derive additional useful information from the existing columns. Two new features, Average Fare and Average Tip, were created to understand the average fare and tip generated per taxi trip.

Average Fare: Represents the average fare amount for each trip.

Average Tip: Represents the average tip amount for each trip.

```
In [47]: import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

```
In [48]: df = pd.read_csv(r"D:\aggregated_nyc_yellow_taxi_2024.csv")
df
```

Out[48]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	c
0	2024-01-01	00:00	1.0	Bronx	Bronx	2	2	1.60	9
1	2024-01-01	00:00	1.0	Bronx	Manhattan	2	1	7.50	1
2	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	1	6	10.28	4
3	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	2	4	5.89	3
4	2024-01-01	00:00	1.0	Brooklyn	Manhattan	1	10	53.13	2
...	...	...	...	...	...	...	...	...	..
921368	2024-12-31	23:00	Unknown	Manhattan	Queens	2	1	14.40	3
921369	2024-12-31	23:00	Unknown	Queens	Brooklyn	0	11	90.42	2
921370	2024-12-31	23:00	Unknown	Queens	Manhattan	0	6	44.88	1
921371	2024-12-31	23:00	Unknown	Queens	Manhattan	1	1	17.00	3
921372	2024-12-31	23:00	Unknown	Queens	Queens	0	16	51.67	2

921373 rows × 18 columns

In [49]:

df.head()

Out[49]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	duration
0	2024-01-01	00:00	1.0	Bronx	Bronx	2	2	1.60	9.2166
1	2024-01-01	00:00	1.0	Bronx	Manhattan	2	1	7.50	18.666
2	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	1	6	10.28	42.750
3	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	2	4	5.89	38.050
4	2024-01-01	00:00	1.0	Brooklyn	Manhattan	1	10	53.13	263.68

In [50]: `df.tail()`

Out[50]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	c
921368	2024-12-31	23:00	Unknown	Manhattan	Queens	2	1	14.40	3
921369	2024-12-31	23:00	Unknown	Queens	Brooklyn	0	11	90.42	2
921370	2024-12-31	23:00	Unknown	Queens	Manhattan	0	6	44.88	1
921371	2024-12-31	23:00	Unknown	Queens	Manhattan	1	1	17.00	3
921372	2024-12-31	23:00	Unknown	Queens	Queens	0	16	51.67	2

In [51]: `df.shape`

Out[51]: (921373, 18)

In [52]: `df.columns`

Out[52]: Index(['date', 'hour', 'passenger_count', 'PU_Borough', 'DO_Borough',
'payment_type', 'trip_count', 'trip_distance_sum', 'duration_sum',
'fare_amount_sum', 'extra_sum', 'mta_tax_sum', 'tip_amount_sum',
'tolls_amount_sum', 'improvement_surcharge_sum',
'congestion_surcharge_sum', 'airport_fee_sum', 'total_amount_sum'],
dtype='str')

```
In [53]: df.dtypes
```

```
Out[53]: date                str
         hour                str
         passenger_count      str
         PU_Borough           str
         DO_Borough           str
         payment_type         int64
         trip_count           int64
         trip_distance_sum    float64
         duration_sum         float64
         fare_amount_sum      float64
         extra_sum            float64
         mta_tax_sum          float64
         tip_amount_sum       float64
         tolls_amount_sum     float64
         improvement_surcharge_sum float64
         congestion_surcharge_sum float64
         airport_fee_sum      float64
         total_amount_sum     float64
         dtype: object
```

In [54]: `df.describe()`

Out[54]:

	payment_type	trip_count	trip_distance_sum	duration_sum	fare_amount_sum	extra_sum	mta_tax
count	921373.000000	921373.000000	921373.000000	921373.000000	921373.000000	921373.000000	921373.00
mean	1.566788	42.740245	143.666461	718.476107	834.065682	61.802906	21.286290
std	1.058372	270.857268	573.559375	3771.042279	3974.910908	495.763885	135.35760
min	0.000000	1.000000	0.010000	0.016667	3.000000	0.000000	0.000000
25%	1.000000	1.000000	8.290000	28.766667	43.500000	0.000000	0.500000
50%	1.000000	3.000000	20.290000	68.233333	100.000000	3.500000	1.000000
75%	2.000000	9.000000	64.700000	243.583333	321.100000	13.000000	4.500000
max	4.000000	7242.000000	11781.520000	112972.700000	105949.340000	22058.000000	3619.5000



In [55]: df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 921373 entries, 0 to 921372
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   date                                  921373 non-null  str
1   hour                                  921373 non-null  str
2   passenger_count                       921373 non-null  str
3   PU_Borough                            921373 non-null  str
4   DO_Borough                            921373 non-null  str
5   payment_type                          921373 non-null  int64
6   trip_count                           921373 non-null  int64
7   trip_distance_sum                    921373 non-null  float64
8   duration_sum                         921373 non-null  float64
9   fare_amount_sum                      921373 non-null  float64
10  extra_sum                            921373 non-null  float64
11  mta_tax_sum                          921373 non-null  float64
12  tip_amount_sum                       921373 non-null  float64
13  tolls_amount_sum                     921373 non-null  float64
14  improvement_surcharge_sum            921373 non-null  float64
15  congestion_surcharge_sum             921373 non-null  float64
16  airport_fee_sum                      921373 non-null  float64
17  total_amount_sum                     921373 non-null  float64
dtypes: float64(11), int64(2), str(5)
memory usage: 126.5 MB
```

```
In [56]: df.isna().sum()
```

```
Out[56]: date                0
         hour                0
         passenger_count      0
         PU_Borough          0
         DO_Borough          0
         payment_type        0
         trip_count          0
         trip_distance_sum    0
         duration_sum         0
         fare_amount_sum      0
         extra_sum            0
         mta_tax_sum          0
         tip_amount_sum       0
         tolls_amount_sum     0
         improvement_surcharge_sum 0
         congestion_surcharge_sum 0
         airport_fee_sum      0
         total_amount_sum     0
         dtype: int64
```

```
In [57]: df.duplicated().sum()
```

```
Out[57]: np.int64(0)
```

```
In [58]: df.nunique()
```

```
Out[58]: date                366  
         hour                24  
         passenger_count      6  
         PU_Borough          7  
         DO_Borough          7  
         payment_type         5  
         trip_count          4175  
         trip_distance_sum    91407  
         duration_sum        143340  
         fare_amount_sum     112094  
         extra_sum           10409  
         mta_tax_sum          4225  
         tip_amount_sum      78232  
         tolls_amount_sum    12886  
         improvement_surcharge_sum 4257  
         congestion_surcharge_sum  4062  
         airport_fee_sum       453  
         total_amount_sum    204537  
         dtype: int64
```

```
In [59]: df['payment_type'].value_counts()
```

```
Out[59]: payment_type  
1      415283  
2      270211  
0      101089  
4       83521  
3       51269  
Name: count, dtype: int64
```

```
In [60]: df['passenger_count'].value_counts()
```

```
Out[60]: passenger_count
1.0      330093
2.0      184190
Unknown   169149
3.0      102077
4.0       87841
5.0       48023
Name: count, dtype: int64
```

```
In [61]: df.groupby('payment_type')['trip_distance_sum'].mean().reset_index()
```

```
Out[61]:
```

	payment_type	trip_distance_sum
0	0	119.395200
1	1	243.578320
2	2	63.944299
3	3	9.317346
4	4	16.651832

```
In [62]: df.groupby('payment_type') ['fare_amount_sum'].sum().reset_index()
```

Out[62]:

	payment_type	fare_amount_sum
0	0	7.562958e+07
1	1	5.815389e+08
2	2	1.003925e+08
3	3	2.886070e+06
4	4	8.038504e+06

```
In [63]: df.groupby('payment_type') ['fare_amount_sum'].max().reset_index()
```

Out[63]:

	payment_type	fare_amount_sum
0	0	78311.82
1	1	105949.34
2	2	12370.40
3	3	745.10
4	4	1412.50

```
In [64]: df.groupby('passenger_count') ['fare_amount_sum'].mean().reset_index()
```

Out[64]:

	passenger_count	fare_amount_sum
0	1.0	1573.156326
1	2.0	618.024683
2	3.0	252.844308
3	4.0	201.538088
4	5.0	123.282154
5	Unknown	508.016407

```
In [65]: df.groupby('passenger_count') ['trip_count'].sum().reset_index()
```

Out[65]:

	passenger_count	trip_count
0	1.0	27684978
1	2.0	5208779
2	3.0	1221066
3	4.0	756608
4	5.0	314758
5	Unknown	4193519

```
In [66]: df['payment_type'].value_counts('trip_count ').reset_index()
```

Out[66]:

	payment_type	proportion
0	1	0.450722
1	2	0.293270
2	0	0.109716
3	4	0.090648
4	3	0.055644

```
In [67]: df.sort_values(by='trip_distance_sum',ascending=False).head(5)
```

Out[67]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	
872036	2024-12-12	21:00	1.0	Manhattan	Manhattan	1	5513	11781.52	{
801448	2024-11-14	18:00	1.0	Manhattan	Manhattan	1	6658	11733.37	{
816688	2024-11-20	21:00	1.0	Manhattan	Manhattan	1	5413	11721.53	{
866978	2024-12-10	20:00	1.0	Manhattan	Manhattan	1	6160	11636.84	{
869466	2024-12-11	21:00	1.0	Manhattan	Manhattan	1	5344	11621.90	{

```
In [68]: df[df['trip_distance_sum']>100]
```

Out[68]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	c
8	2024-01-01	00:00	1.0	Manhattan	Bronx	1	14	101.03	2
11	2024-01-01	00:00	1.0	Manhattan	Brooklyn	1	83	550.58	2
14	2024-01-01	00:00	1.0	Manhattan	Manhattan	1	2392	5367.68	3
15	2024-01-01	00:00	1.0	Manhattan	Manhattan	2	477	983.47	6
18	2024-01-01	00:00	1.0	Manhattan	Queens	1	44	295.85	1
...	...	...	...	...	...	...	...	...	..
921321	2024-12-31	23:00	3.0	Queens	Manhattan	1	11	147.88	3
921331	2024-12-31	23:00	4.0	Manhattan	Manhattan	1	112	184.04	1
921338	2024-12-31	23:00	4.0	Queens	Manhattan	1	11	138.51	2
921358	2024-12-31	23:00	Unknown	Brooklyn	Brooklyn	0	32	118.93	6
921363	2024-12-31	23:00	Unknown	Manhattan	Manhattan	0	176	599.84	2

170494 rows × 18 columns



```
In [69]: df[(df['trip_distance_sum']>50) & (df['fare_amount_sum']>500)]
```

Out[69]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	c
11	2024-01-01	00:00	1.0	Manhattan	Brooklyn	1	83	550.58	2
14	2024-01-01	00:00	1.0	Manhattan	Manhattan	1	2392	5367.68	3
15	2024-01-01	00:00	1.0	Manhattan	Manhattan	2	477	983.47	6
18	2024-01-01	00:00	1.0	Manhattan	Queens	1	44	295.85	1
19	2024-01-01	00:00	1.0	Manhattan	Queens	2	24	157.45	6
...	...	...	...	...	...	...	...	...	..
921313	2024-12-31	23:00	3.0	Manhattan	Manhattan	1	103	207.49	1
921321	2024-12-31	23:00	3.0	Queens	Manhattan	1	11	147.88	3
921331	2024-12-31	23:00	4.0	Manhattan	Manhattan	1	112	184.04	1
921338	2024-12-31	23:00	4.0	Queens	Manhattan	1	11	138.51	2
921363	2024-12-31	23:00	Unknown	Manhattan	Manhattan	0	176	599.84	2

169898 rows × 18 columns



```
In [70]: df['payment_type'].unique()
```

```
Out[70]: array([2, 1, 4, 3, 0])
```

```
In [71]: df['average_fare'] = df['fare_amount_sum'] / df['trip_count']  
df.head()
```

```
Out[71]:
```

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	duration
0	2024-01-01	00:00	1.0	Bronx	Bronx	2	2	1.60	9.2166
1	2024-01-01	00:00	1.0	Bronx	Manhattan	2	1	7.50	18.666
2	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	1	6	10.28	42.750
3	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	2	4	5.89	38.050
4	2024-01-01	00:00	1.0	Brooklyn	Manhattan	1	10	53.13	263.68

```
In [72]: df['average_tip'] = df['tip_amount_sum']/df['trip_count']  
df.head()
```

Out[72]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	duration
0	2024-01-01	00:00	1.0	Bronx	Bronx	2	2	1.60	9.21667
1	2024-01-01	00:00	1.0	Bronx	Manhattan	2	1	7.50	18.6667
2	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	1	6	10.28	42.7500
3	2024-01-01	00:00	1.0	Brooklyn	Brooklyn	2	4	5.89	38.0500
4	2024-01-01	00:00	1.0	Brooklyn	Manhattan	1	10	53.13	263.680

```
In [73]: df[['average_fare', 'average_tip']].head()
```

Out[73]:

	average_fare	average_tip
0	6.500000	0.000000
1	32.400000	0.000000
2	10.116667	1.941667
3	17.650000	0.000000
4	30.300000	6.720000

```
In [74]: df.sort_values(by='average_fare',ascending = False).head()
```

Out[74]:

	date	hour	passenger_count	PU_Borough	DO_Borough	payment_type	trip_count	trip_distance_sum	
844034	2024-12-01	16:00	4.0	Queens	Queens	4	1	0.07	(
323821	2024-05-13	16:00	4.0	Queens	Unknown	1	1	35.97	9
311174	2024-05-08	19:00	2.0	Queens	Unknown	1	1	27.38	(
411469	2024-06-16	14:00	Unknown	Unknown	Unknown	1	1	1.90	.
784591	2024-11-07	20:00	4.0	Queens	Unknown	1	1	34.11	!



```
In [75]: df.groupby('date') ['trip_count'].sum().reset_index().sort_values(by='trip_count',ascending=False).head(1)
```

Out[75]:

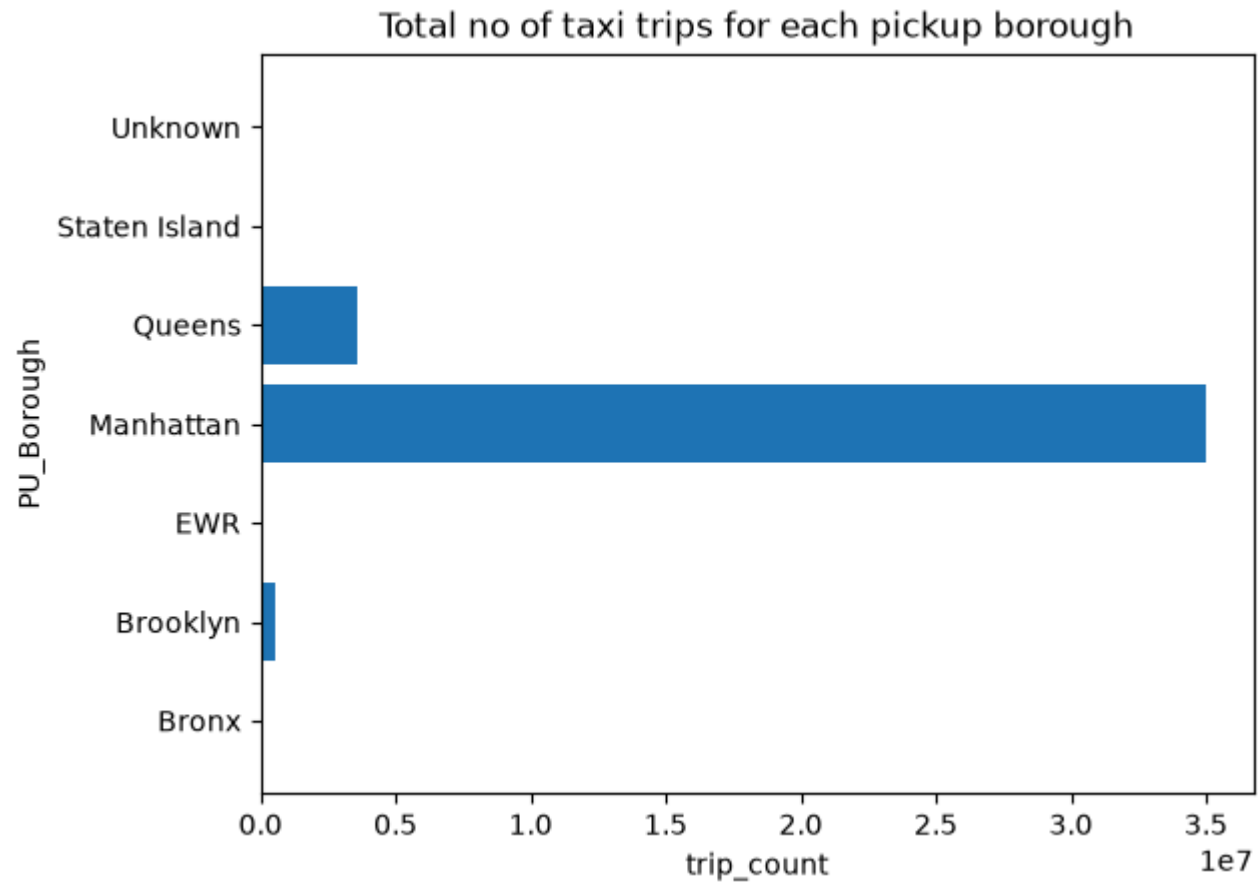
	date	trip_count
348	2024-12-14	153026

```
In [76]: df1 = df.groupby('PU_Borough') ['trip_count'].sum().reset_index()  
df1
```

Out[76]:

	PU_Borough	trip_count
0	Bronx	112899
1	Brooklyn	544612
2	EWB	1046
3	Manhattan	35000569
4	Queens	3599033
5	Staten Island	1040
6	Unknown	120509

```
In [77]: plt.barh(df1['PU_Borough'],df1['trip_count'])
plt.xlabel('trip_count')
plt.ylabel('PU_Borough')
plt.title('Total no of taxi trips for each pickup borough')
plt.show()
```



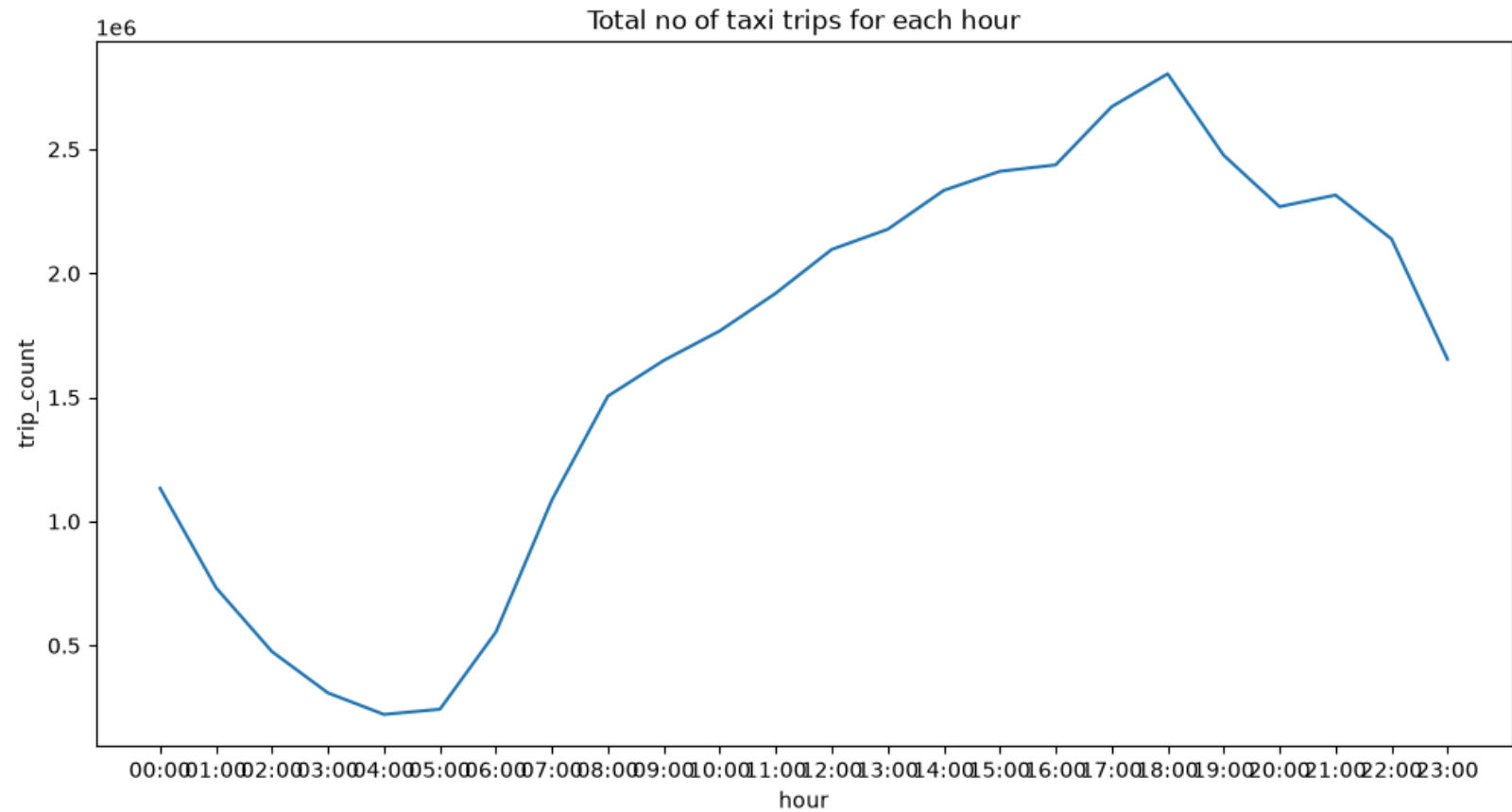
```
In [78]: df2 = df.groupby('hour') ['trip_count'].sum().reset_index()  
df2
```

Out[78]:

	hour	trip_count
0	00:00	1133584
1	01:00	732578
2	02:00	475179
3	03:00	308838
4	04:00	222139
5	05:00	243169
6	06:00	554021
7	07:00	1086563
8	08:00	1504847
9	09:00	1648997
10	10:00	1768025
11	11:00	1919847
12	12:00	2096251
13	13:00	2177377
14	14:00	2334030
15	15:00	2410757
16	16:00	2436792
17	17:00	2671137
18	18:00	2803895

	hour	trip_count
19	19:00	2475955
20	20:00	2268566
21	21:00	2315223
22	22:00	2138456
23	23:00	1653482

```
In [79]: plt.figure(figsize=(12,6))
plt.plot(df2['hour'],df2['trip_count'])
plt.xlabel('hour')
plt.ylabel('trip_count')
plt.title('Total no of taxi trips for each hour')
plt.show()
```



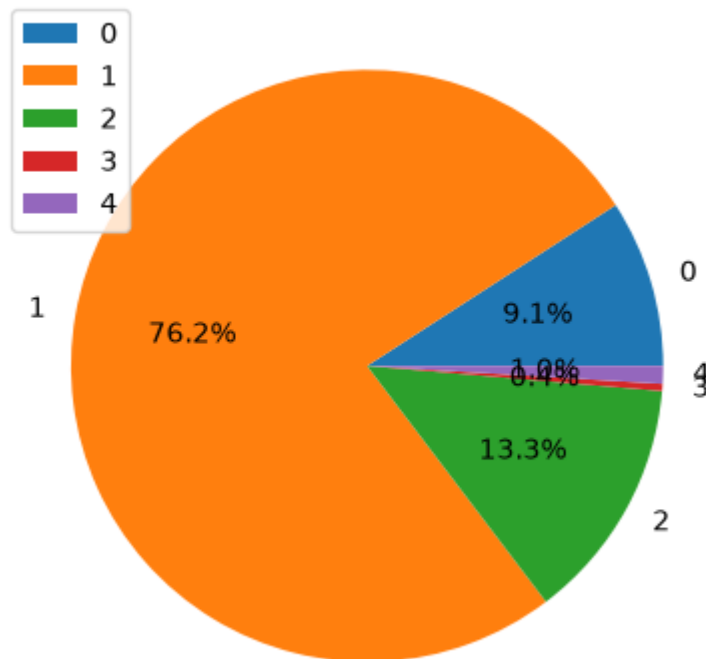
```
In [80]: df3 = df.groupby('payment_type') ['trip_count'].sum().reset_index()  
df3
```

Out[80]:

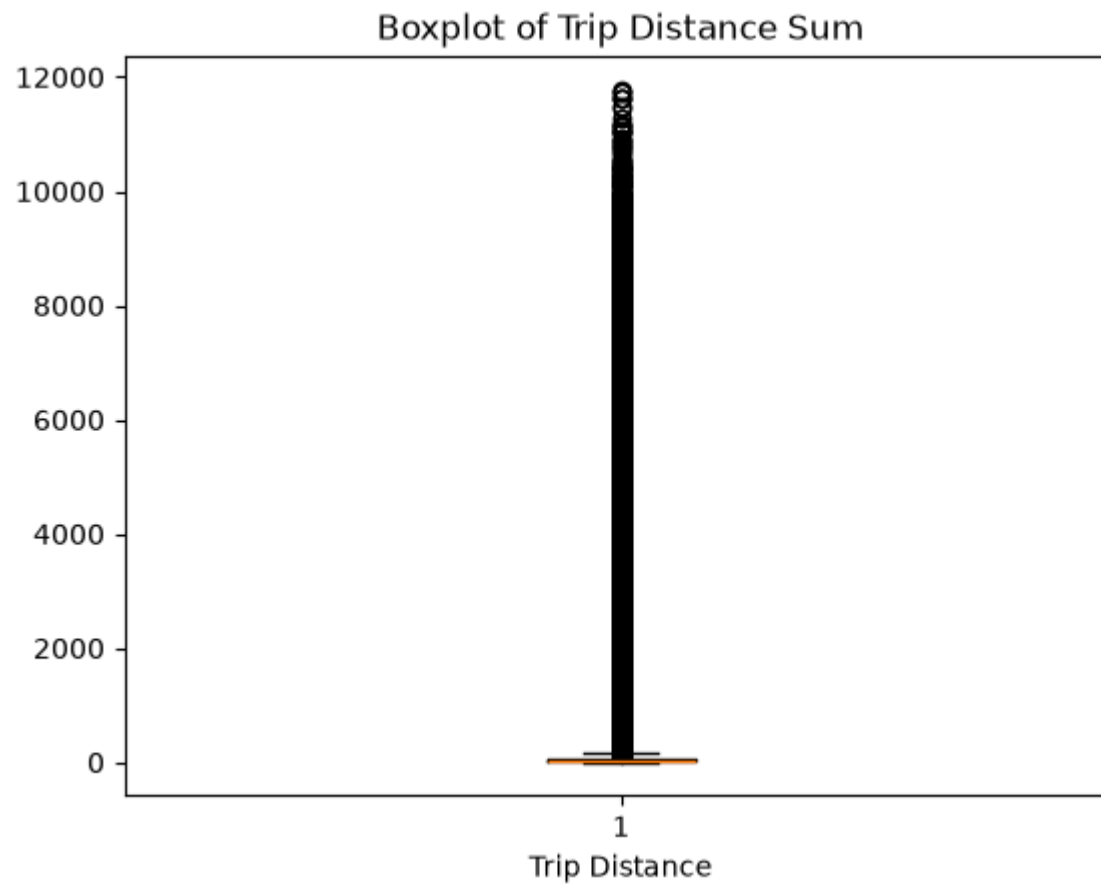
	payment_type	trip_count
0	0	3599108
1	1	30003364
2	2	5244084
3	3	153313
4	4	379839

```
In [81]: plt.pie(df3['trip_count'],labels=df3['payment_type'],autopct='%1.1f%%')  
plt.title('Total no of taxi trips for each payment type')  
plt.legend()  
plt.show()
```

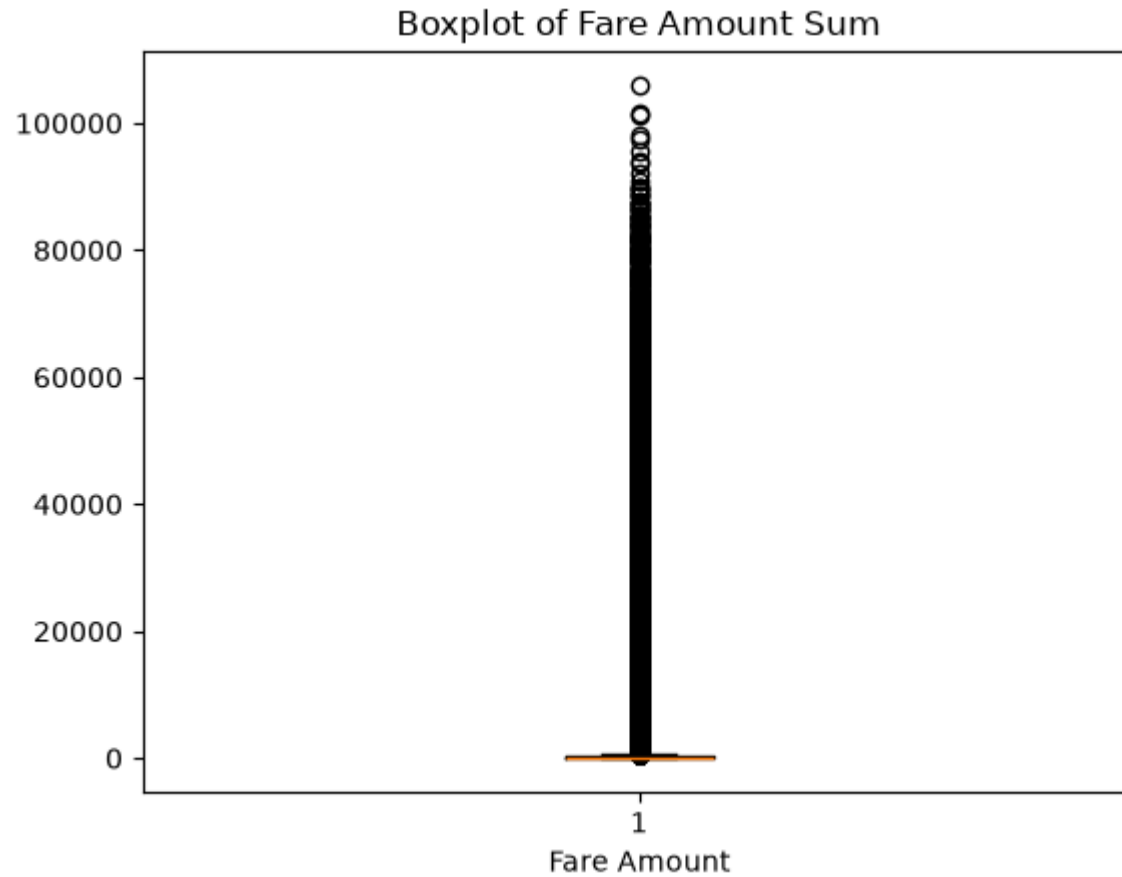
Total no of taxi trips for each payment type



```
In [82]: plt.boxplot(df['trip_distance_sum'])  
plt.xlabel('Trip Distance')  
plt.title('Boxplot of Trip Distance Sum')  
plt.show()
```



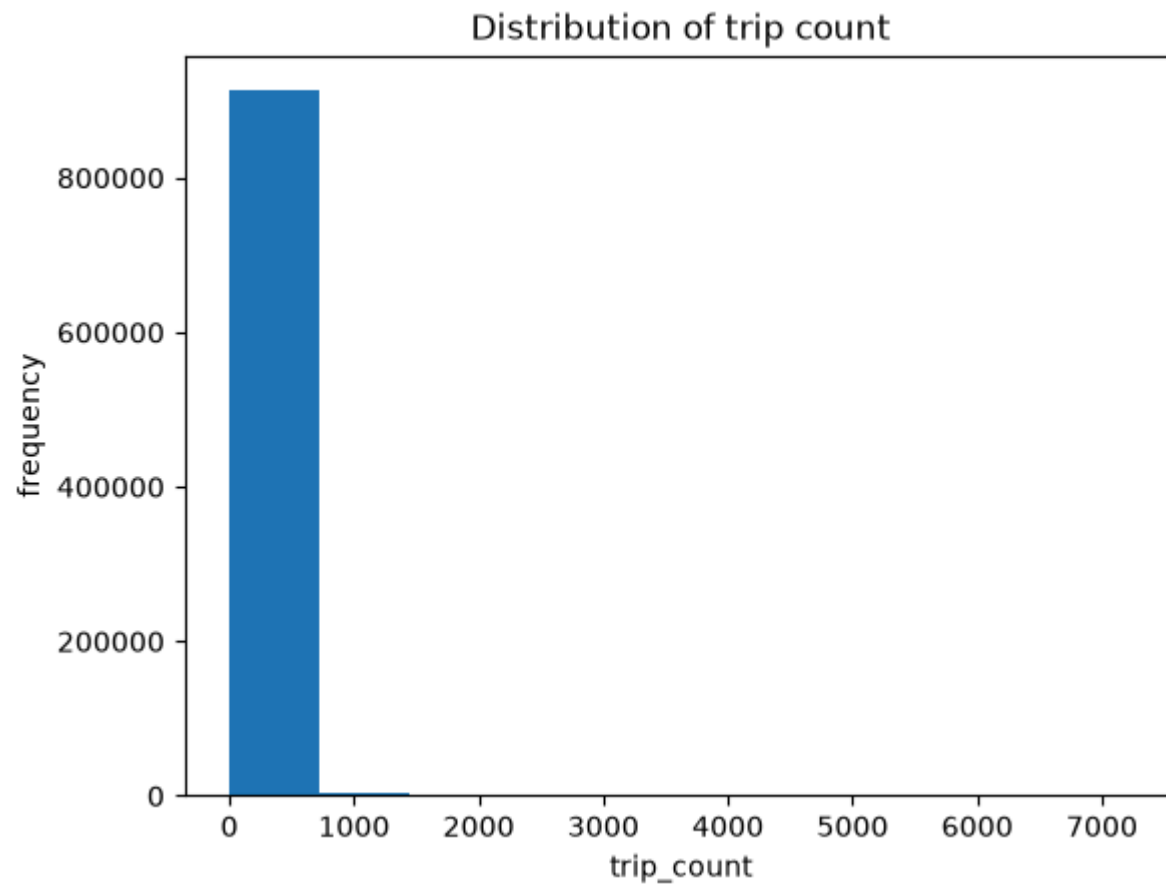
```
In [83]: plt.boxplot(df['fare_amount_sum'])  
plt.xlabel('Fare Amount')  
plt.title('Boxplot of Fare Amount Sum')  
plt.show()
```



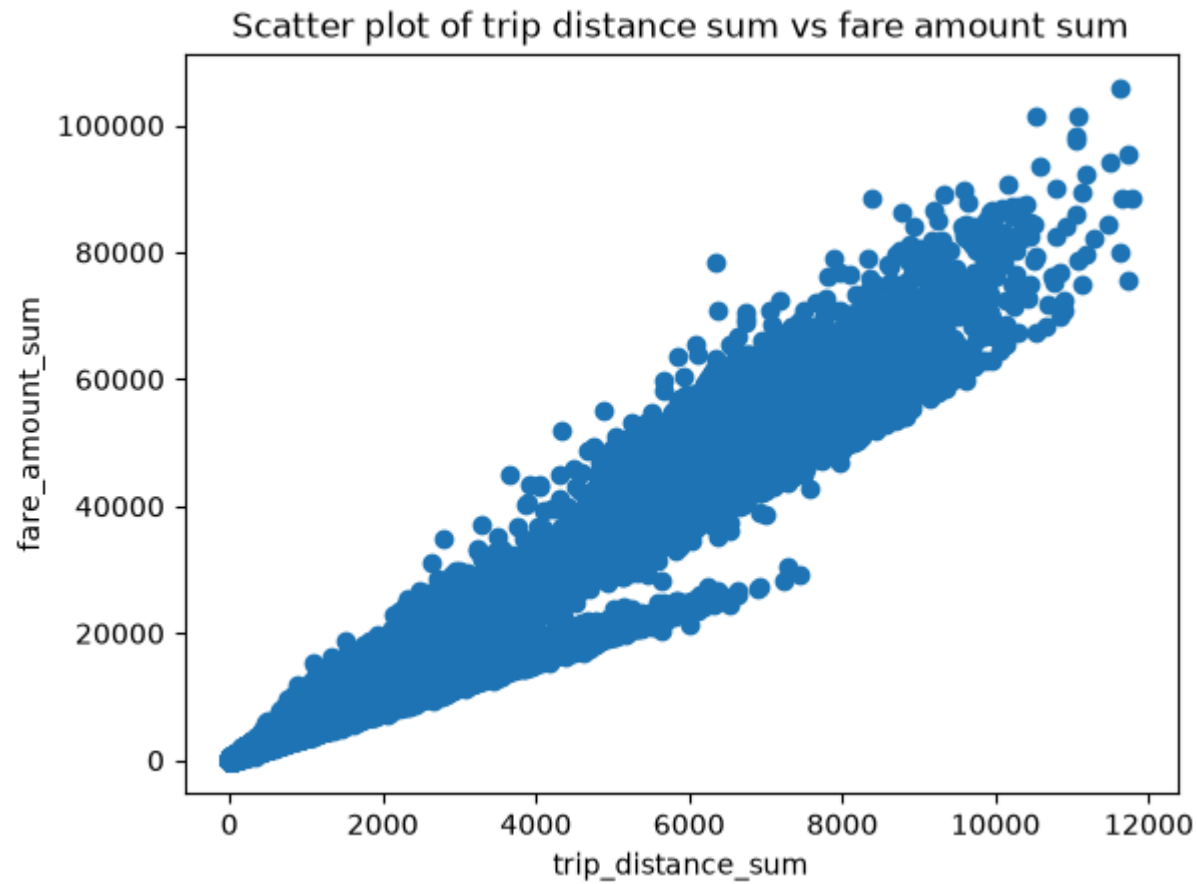
```
In [84]: plt.hist(df['trip_count'])

plt.xlabel('trip_count')
plt.ylabel('frequency')
plt.title('Distribution of trip count')

plt.show()
```

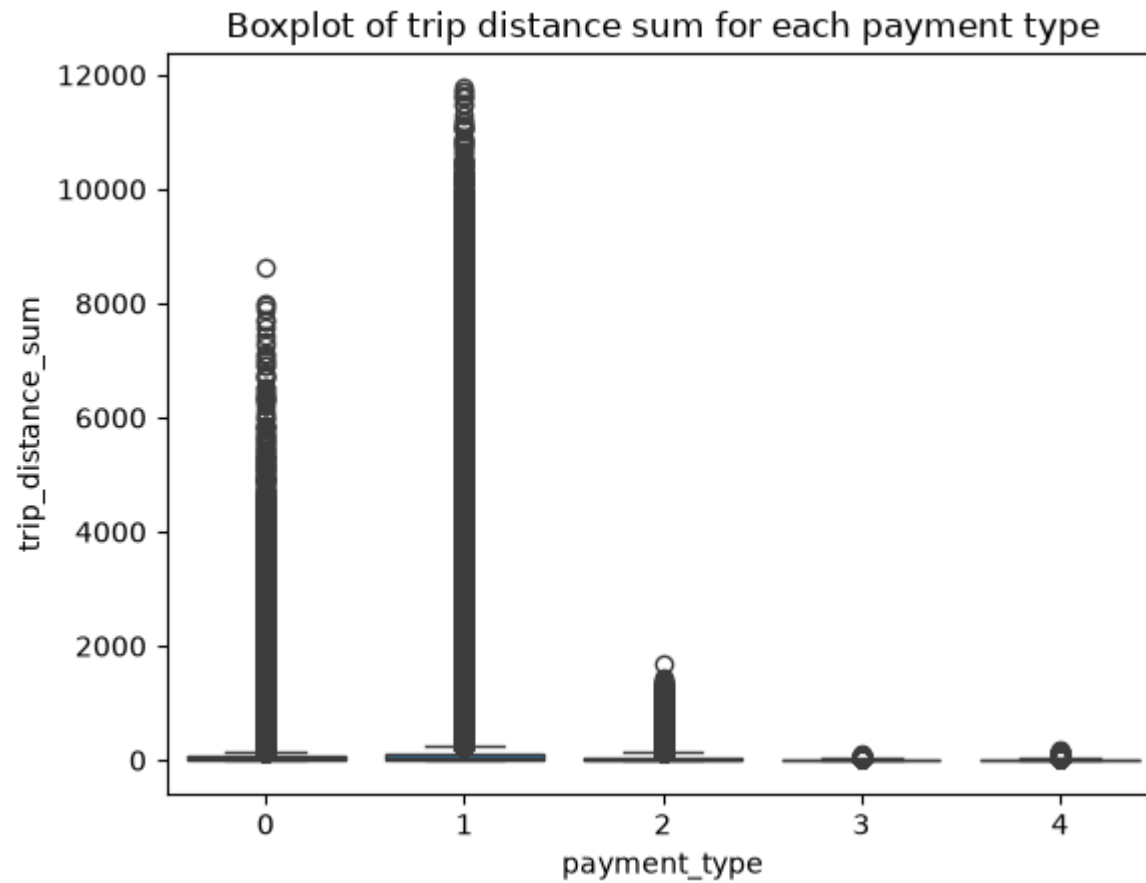


```
In [85]: plt.scatter(df['trip_distance_sum'],df['fare_amount_sum'])  
plt.xlabel('trip_distance_sum')  
plt.ylabel('fare_amount_sum')  
plt.title('Scatter plot of trip distance sum vs fare amount sum')  
plt.show()
```

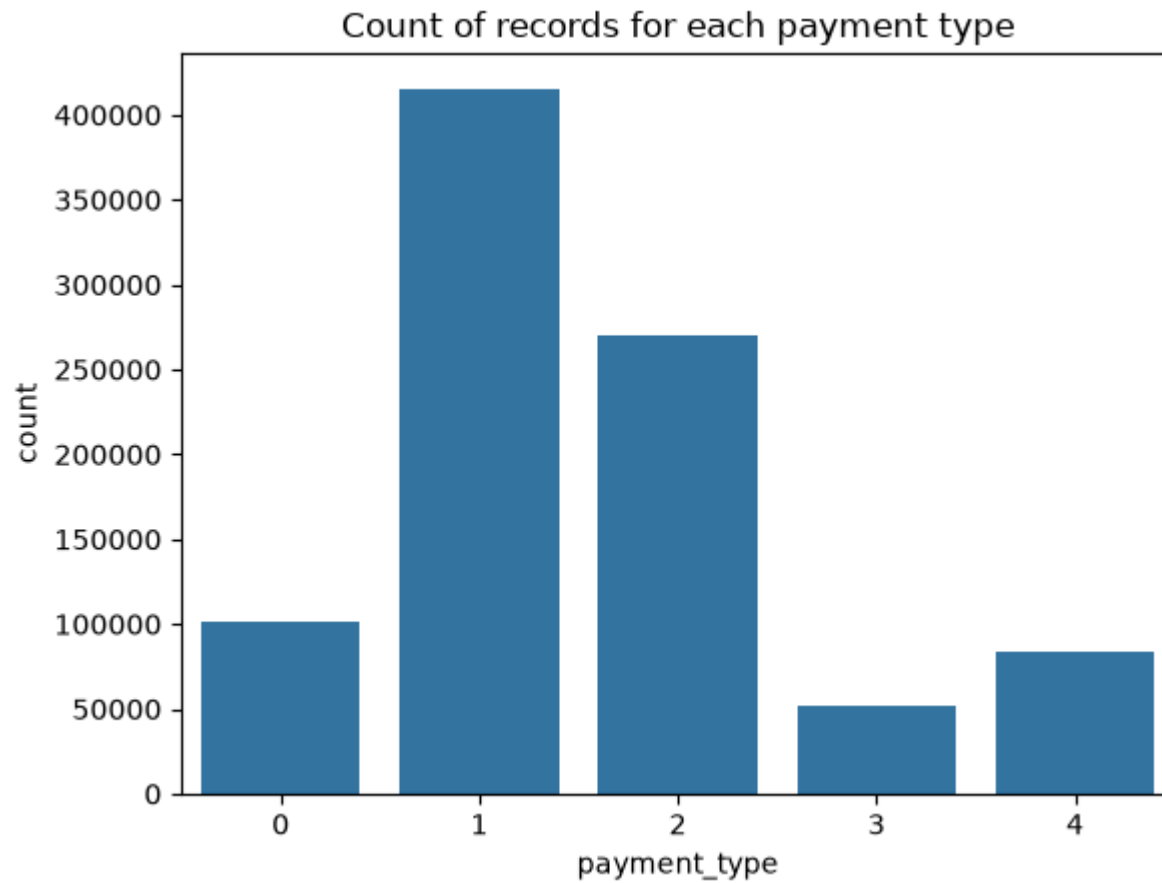


```
In [86]: import seaborn as sns
```

```
In [87]: sns.boxplot(x=df['payment_type'],y=df['trip_distance_sum'])  
plt.xlabel('payment_type')  
plt.ylabel('trip_distance_sum')  
plt.title('Boxplot of trip distance sum for each payment type')  
plt.show()
```



```
In [88]: sns.countplot(x=df['payment_type'])  
plt.xlabel('payment_type')  
plt.ylabel('count')  
plt.title('Count of records for each payment type')  
plt.show()
```

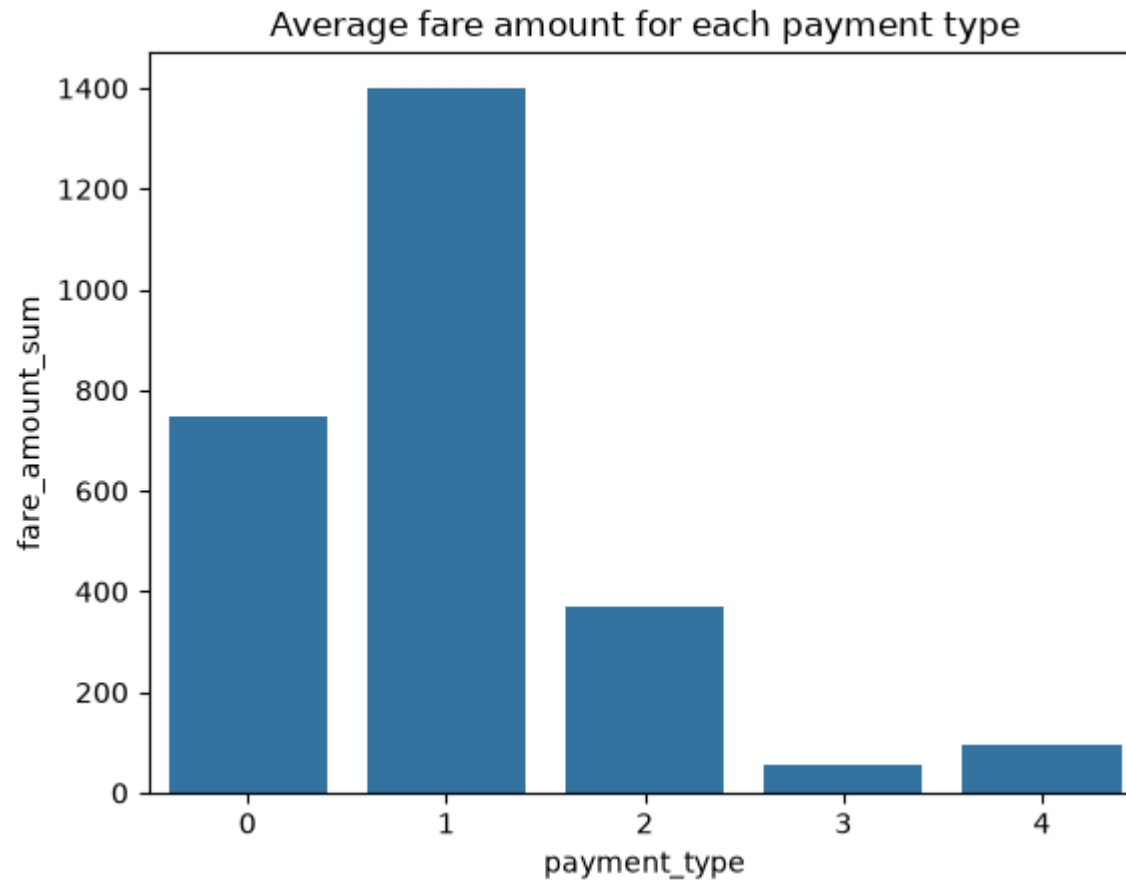


```
In [89]: df4= df.groupby('payment_type') ['fare_amount_sum'].mean().reset_index()  
df4
```

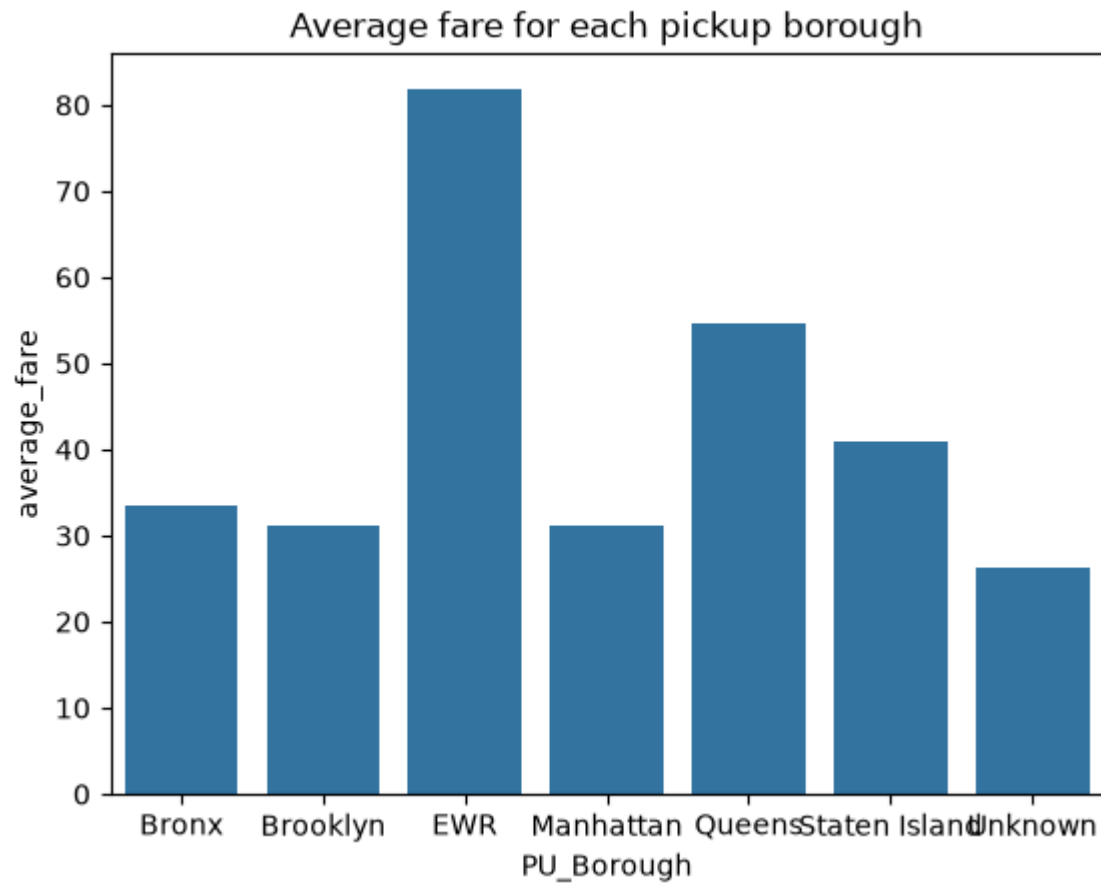
Out[89]:

	payment_type	fare_amount_sum
0	0	748.148473
1	1	1400.343741
2	2	371.533707
3	3	56.292692
4	4	96.245304

```
In [90]: sns.barplot(x=df4['payment_type'],y=df4['fare_amount_sum'])  
plt.xlabel('payment_type')  
plt.ylabel('fare_amount_sum')  
plt.title('Average fare amount for each payment type')  
plt.show()
```



```
In [91]: df5 = df.groupby('PU_Borough') ['average_fare'].mean().reset_index()
df5
sns.barplot(x=df5['PU_Borough'],y=df5['average_fare'])
plt.xlabel('PU_Borough')
plt.ylabel('average_fare')
plt.title('Average fare for each pickup borough')
plt.show()
```



```
In [92]: passenger_distribution = df['passenger_count'].value_counts().reset_index()

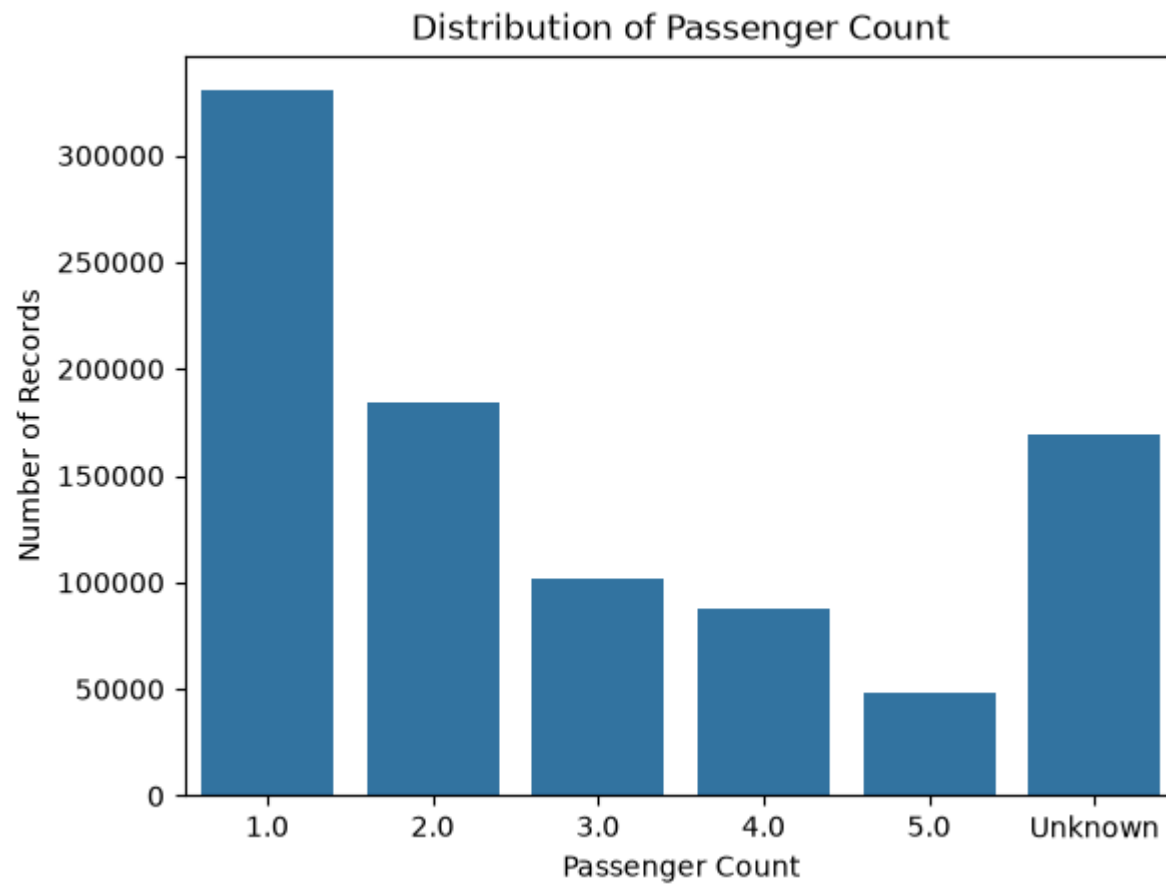
passenger_distribution
```

Out[92]:

	passenger_count	count
0	1.0	330093
1	2.0	184190
2	Unknown	169149
3	3.0	102077
4	4.0	87841
5	5.0	48023

```
In [93]: sns.countplot(x=df['passenger_count'])

plt.xlabel('Passenger Count')
plt.ylabel('Number of Records')
plt.title('Distribution of Passenger Count')
plt.show()
```

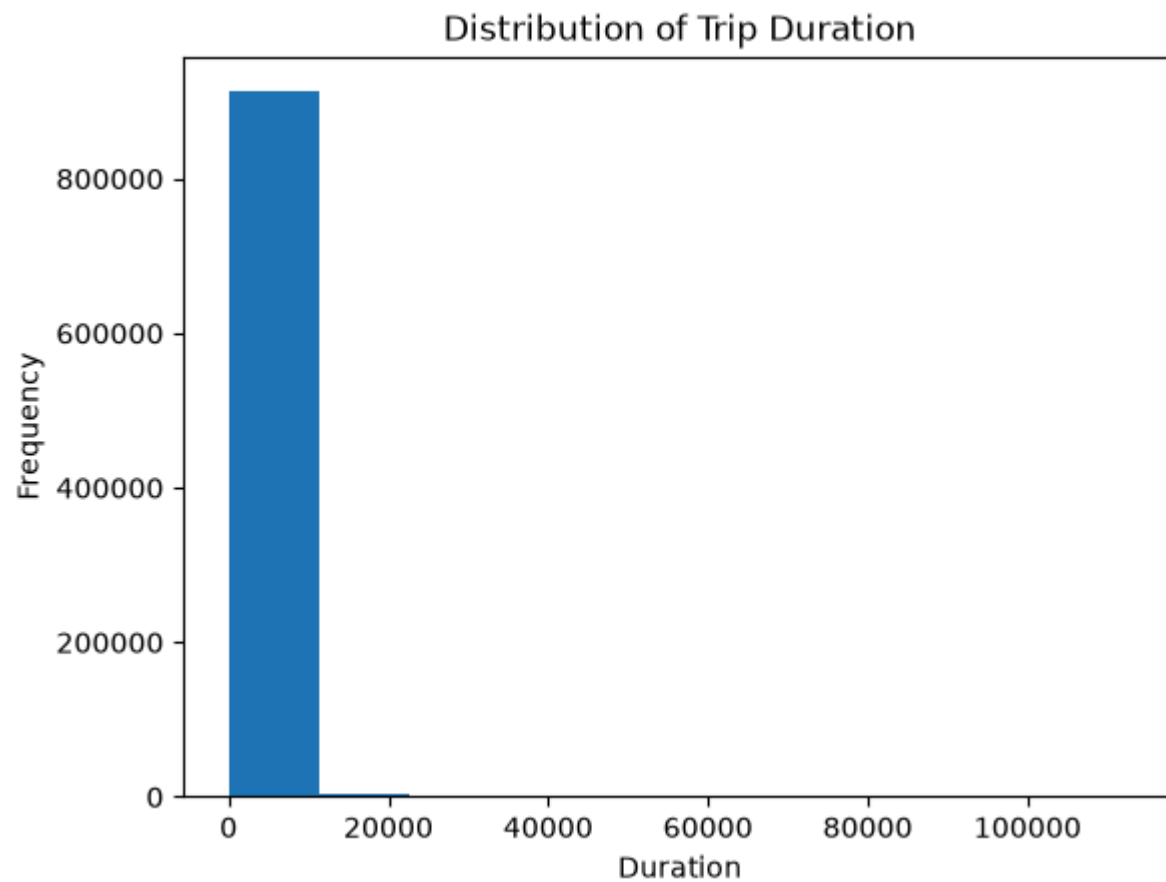


```
In [94]: df['date'].value_counts().reset_index().head(10)
```

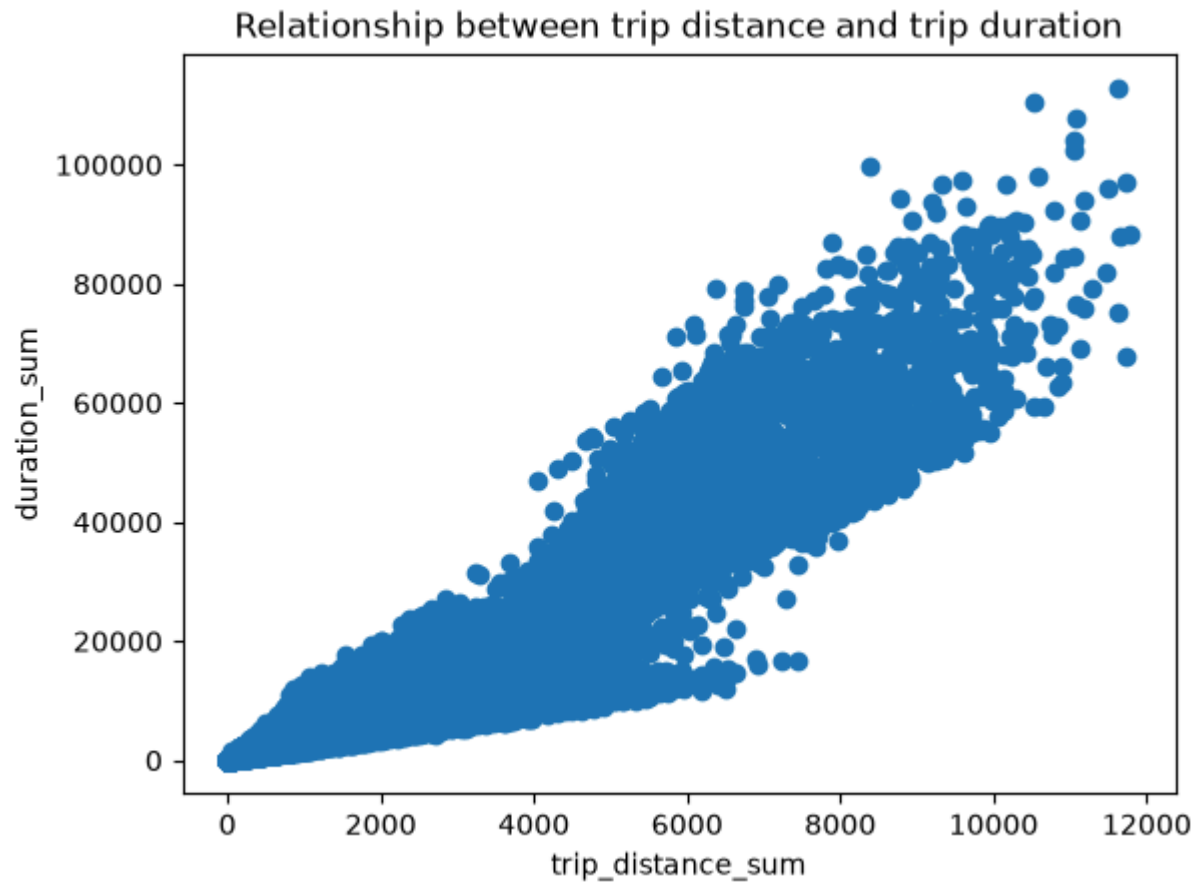
Out[94]:

	date	count
0	2024-09-01	2951
1	2024-10-27	2915
2	2024-08-31	2898
3	2024-10-26	2867
4	2024-08-24	2864
5	2024-07-21	2859
6	2024-08-25	2854
7	2024-10-12	2843
8	2024-06-15	2830
9	2024-06-16	2830

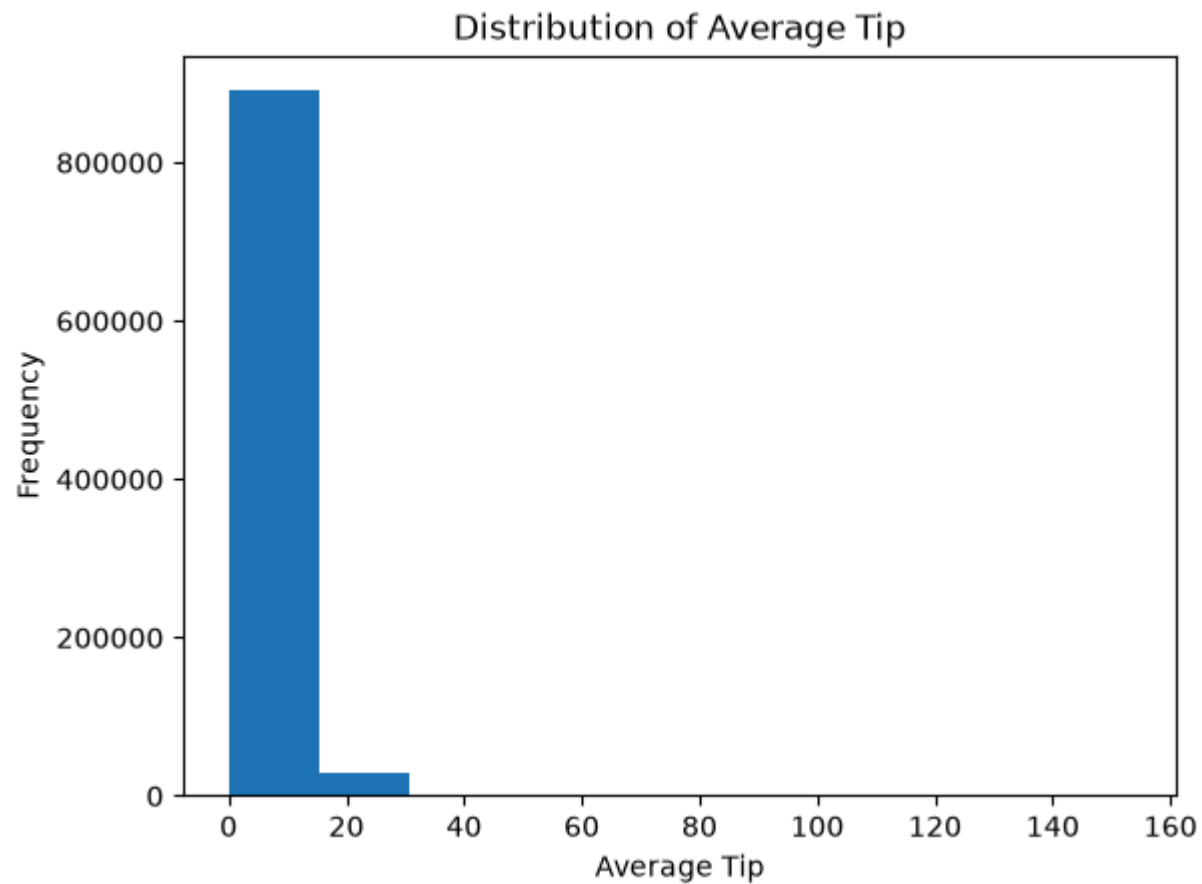
```
In [95]: df['duration_sum'].describe()
plt.hist(df['duration_sum'])
plt.xlabel('Duration')
plt.ylabel('Frequency')
plt.title('Distribution of Trip Duration')
plt.show()
```



```
In [96]: plt.scatter(df['trip_distance_sum'],df['duration_sum'])  
plt.xlabel('trip_distance_sum')  
plt.ylabel('duration_sum')  
plt.title('Relationship between trip distance and trip duration')  
plt.show()
```



```
In [97]: df['average_tip'].describe()  
plt.hist(df['average_tip'])  
plt.xlabel('Average Tip')  
plt.ylabel('Frequency')  
plt.title('Distribution of Average Tip')  
plt.show()
```



Overall Key Insights

- * NYC Yellow Taxi usage varies across different hours of the day, showing clear differences in trip activity throughout the day.
- * Pickup activity is not evenly distributed across the NYC boroughs, with some boroughs recording more taxi trips than others.
- * The analysis shows that payment methods are not equally used, indicating differences in passenger payment preferences.
- * Passenger counts show that taxi trips are more common for certain passenger group sizes.
- * Trip distance and fare amount show a relationship, indicating that longer taxi trips generally involve higher fare amounts.
- * Trip duration and trip distance also show a relationship, helping to understand how travel distance is associated with the time taken for trips.
- * The boxplot analysis identifies variation and potential outliers in trip distance and fare amounts.
- * Average fare and average tip provide additional information about the amount generated per taxi trip.
- * Overall, the analysis provides useful insights into taxi usage, passenger behavior, payment preferences, trip characteristics, and fare patterns in NYC during 2024.

CONCLUSION

This analysis provides valuable insights into NYC Yellow Taxi trip data for the year 2024. The study helps understand taxi usage patterns based on trip hours, pickup boroughs, passenger count, payment methods, trip distance, trip duration, and fare amounts. The analysis also highlights relationships between trip distance, duration, and fare amount. The findings provide a better understanding of passenger behavior, taxi usage patterns, and fare characteristics in NYC.

- * Taxi trips vary across different hours of the day.
- * Some pickup boroughs have more taxi trips than others.
- * Payment methods show different usage patterns.
- * Trip distance and trip duration have a relationship.
- * Longer trips generally have higher fare amounts.
- * The analysis identifies variations and outliers in trip distance and fare amounts.

Exported with [runcell](https://www.runcell.dev) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://www.runcell.dev).